SISTEMAS DISTRIBUÍDOS

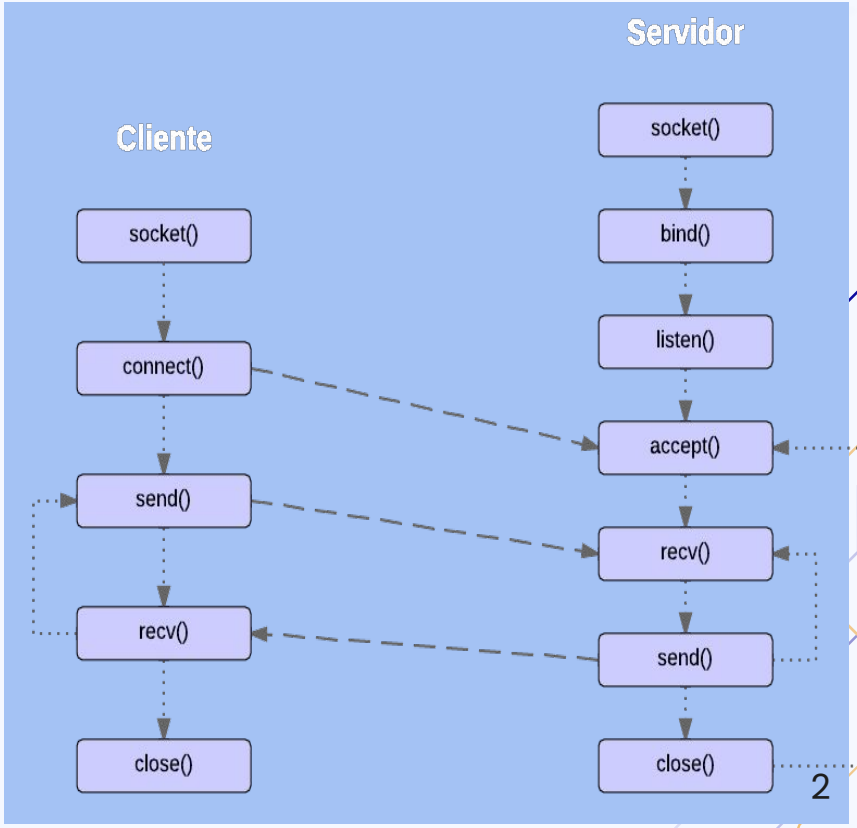
Comunicação

Prof Priscila Marques Kai

LIMITAÇÕES DOS SOCKETS

Sockets eram considerados insuficientes porque:

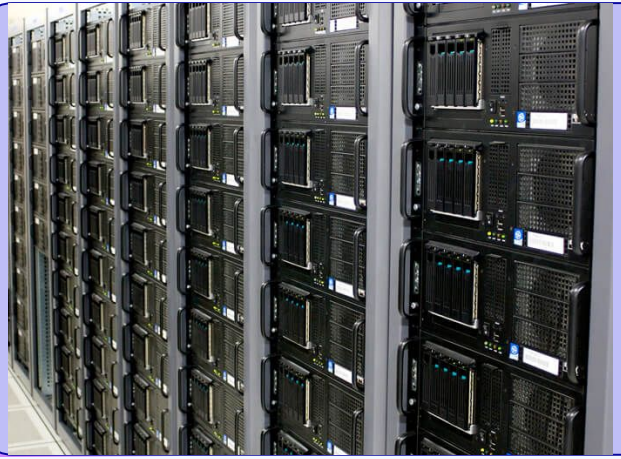
- **Oferecem baixo nível de abstração:** a aplicação precisa lidar diretamente com detalhes de comunicação (mensagens, buffers, endereços, controle de erros), basicamente usando send e recv, o que aumenta a complexidade do código.



LIMITAÇÕES DOS SOCKETS

Sockets eram considerados insuficientes porque:

- **Foram projetados para redes TCP/IP "genéricas":** em **clusters** e ambientes de alto desempenho, redes de interconexão (com latência muito baixa e alto throughput) costumam exigir mecanismos mais específicos e eficientes do que a pilha TCP/IP tradicional.



Conjunto de computadores (nós) conectados por rede e configurados para cooperar.
Normalmente existe:

- vários **nós de processamento** (onde o trabalho roda),
- e, às vezes, nós de **armazenamento** e **gerência** (controle/monitoramento).

LIMITAÇÕES DOS SOCKETS

Sockets eram considerados insuficientes porque:

- **Foram projetados para redes TCP/IP "genéricas":** em clusters e **ambientes de alto desempenho**, redes de interconexão (com latência muito baixa e alto throughput) costumam exigir mecanismos mais específicos e eficientes do que a pilha TCP/IP tradicional.

HPC (High Performance Computing) é o uso de hardware e software para obter **máximo desempenho**, principalmente em:

- simulações científicas,
- modelagem numérica,
- análise de grandes volumes de dados,
- treinamento/execução de modelos grandes,
- renderização e engenharia.

LIMITAÇÕES DOS SOCKETS

Sockets eram considerados insuficientes porque:

- **Podem introduzir sobrecarga desnecessária:** camadas do TCP/IP e cópias de dados na pilha de rede podem limitar o desempenho quando a meta é extrair o máximo da interconexão de alta velocidade.

COMUNICAÇÃO PROPRIETÁRIA

- Para contornar as limitações, os fabricantes passaram a oferecer **bibliotecas proprietárias de comunicação**, voltadas ao alto desempenho.
- Essas bibliotecas traziam **operações mais abstratas** (de nível mais alto) e **otimizadas**, explorando melhor o hardware e a rede.
- Problema: **incompatibilidade** → dificultava a portabilidade de aplicações entre plataformas diferentes.

MPI (MESSAGE-PASSING INTERFACE)

- O **MPI (Message Passing Interface)** foi definido como um **padrão portátil**, ou seja, funciona de forma consistente em diferentes hardwares e sistemas.

- Ele é voltado para **aplicações paralelas**, em que processos trocam mensagens durante a execução. A comunicação é **transiente**: as mensagens não ficam armazenadas para uso futuro (não é comunicação "persistente").

MPI (MESSAGE-PASSING INTERFACE)

- Para obter **alto desempenho**, o MPI explora de forma eficiente a **rede/interconexão disponível** no ambiente (quando existe hardware especializado, ele se beneficia disso).

- Em geral, o MPI assume um modelo em que falhas graves (queda de processo, nó, ou rede) são **fatais para a execução**: normalmente não há um mecanismo automático de recuperação embutido no modelo tradicional.

MPI (MESSAGE-PASSING INTERFACE)

- Em vez de usar endereços **IP + portas**, MPI trabalha com **grupos de processos**:
- Cada **grupo** possui um identificador (o *groupID*/comunicador).
- Dentro de um grupo, cada processo recebe um **rank** (pode pensar como *processID*).
- Assim, o par **(grupo, rank)** identifica de forma única quem envia e quem recebe mensagens naquele contexto.

Vários grupos na mesma aplicação

- Uma aplicação pode criar múltiplos grupos durante a execução.
- Esses grupos podem coexistir e até se sobrepor (um mesmo processo pode participar de mais de um grupo).

MPI — comunicação por troca de mensagens (transiente)

- A base do MPI é a **troca de mensagens** entre processos, para comunicação **transiente** (as mensagens existem apenas durante a execução).

  Operações principais

  - `MPI_Send`: envia uma mensagem para outro processo (destino).
  - `MPI_Recv`: recebe uma mensagem de outro processo (origem).

MPI — comunicação por troca de mensagens (transiente)

- A base do MPI é a **troca de mensagens** entre processos, para comunicação **transiente** (as mensagens existem apenas durante a execução).

  Há variantes para comunicação para diferentes necessidades:
  - **síncrona** (envio/recebimento mais “acoplados”)
  - **assíncrona / não bloqueante** (permite continuar executando)
  - **com buffer** (uso de buffers para envio)
  - **coletiva** (vários processos participam)

Exemplo com **mpi4py** para envio e recebimento de mensagens com MPI.

```
from mpi4py import MPI
```

Para instalar:

```
!pip install mpi4py
```

`mpi4py` - conceitos básicos

- **Comunicador padrão**: `MPI.COMM_WORLD`
  - Reúne todos os processos iniciados pelo MPI.
- **Rank** (identificador do processo): `comm.Get_rank()`
  - Número inteiro que identifica cada processo (0, 1, 2, ...).
- **Size** (número de processos): `comm.Get_size()`
  - Quantos processos estão executando o programa.

Exemplo com **mpi4py** para envio e recebimento de mensagens com MPI.

```
from mpi4py import MPI
```

Para executar, use:

Colab

```
!mpirun --allow-run-as-root --oversubscribe -np
numero_de_processos python -u mpi_exemplo.py
```

VSCode

```
!mpiexec -n numero_de_processos python
mpi_exemplo.py
```

mpi4py - exemplo básico

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()
print('My rank is ',rank)
print('Numero de processos: ',size)
```

```
# execute no terminal com: mpiexec -n 1 mpi_exemplo.py
# saída:
#   My rank is  0
#   Numero de processos:  1
```

Exemplo 2 - Vamos implementar o envio Ponto a Ponto

- Processo 0 → envia uma mensagem
- Processo 1 → recebe a mensagem

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

if size < 2:
    raise SystemExit("Execute com pelo menos 2 processos")
```

Exemplo 2 - Vamos implementar o envio Ponto a Ponto

- Processo 0 → envia uma mensagem
- Processo 1 → recebe a mensagem

Operações principais:

- **`MPI_Send, MPI_Recv`**
- Operações coletivas (**`bcast, scatter, gather`**).

Exemplo 2 - Vamos implementar o envio Ponto a Ponto

- Processo 0 → envia uma mensagem
- Processo 1 → recebe a mensagem

```
if rank == 0:
    comm.send("Olá, processo 1!", dest=1)
elif rank == 1:
    msg = comm.recv(source=0)
    print("Mensagem recebida:", msg)
```

Exemplo 2 - Vamos implementar o envio Ponto a Ponto

- Processo 0 → envia uma mensagem
- Processo 1 → recebe a mensagem
- Com uso do parâmetro **tag**

```
if rank == 0:
    msg = {"texto": "Olá do rank 0!", "valor": 42}
    comm.send(msg, dest=1, tag=10)
    print("Rank 0: enviei a mensagem para o rank 1")
elif rank == 1:
    recebido = comm.recv(source=0, tag=10)
    print(f"Rank 1: recebi -> {recebido}")
```

Exemplo 2 - Vamos implementar o envio Ponto a Ponto

- Com uso do parâmetro **tag**

```
if rank == 0:
    msg = {"texto": "Olá do rank 0!", "valor": 42}
    comm.send(msg, dest=1, tag=10)
    print("Rank 0: enviei a mensagem para o rank 1")
elif rank == 1:
    recebido = comm.recv(source=0, tag=10)
    print(f"Rank 1: recebi -> {recebido}")
```

```
Saída:
    Rank 0: enviei a mensagem para o rank 1
    Rank 1: recebi -> {'texto': 'Ola do rank 0!', 'valor': 42}
```

Exemplo 3 - Envio em anel

- A ideia é fazer com que cada processo envie uma mensagem para o próximo.
- O último processo envia para o primeiro.

```
dest = (rank + 1) % size
src  = (rank - 1) % size

comm.send(f"Mensagem do {rank}", dest=dest)
msg = comm.recv(source=src)
print(f"[{rank}] recebeu: {msg}")
```

Exemplo 3 - Envio em anel

```
dest = (rank + 1) % size
src  = (rank - 1) % size

comm.send(f"Mensagem do {rank}", dest=dest)
msg = comm.recv(source=src)
print(f"[{rank}] recebeu: {msg}")
```

Saída:

```
[1] recebeu: Mensagem do 0
[2] recebeu: Mensagem do 1
[3] recebeu: Mensagem do 2
[0] recebeu: Mensagem do 3
```

Exemplo 4 - Implementação com **Broadcast**

- Nesse exemplo teremos um processo enviando uma mensagem para todos os outros processos.

```
data = None
if rank == 0:
    data = "Olá vizinhos!"
data = comm.bcast(data, root=0)
print(f"[{rank}] recebeu: {data}")
```

ATIVIDADE 1 - Implemente um programa onde o processo 0 envia uma mensagem para o processo 1, que por sua vez responde com outra mensagem para o processo 0.

Para isso, use:

```
comm.send(mensagem, dest=rank_do_processo_destinatario)
resposta = comm.recv(source=rank_do_processo_remetente)
```

MPI – DISCUSSÃO

- **Ponto a ponto:** simples, mas limitado.
- **Anel:** útil para token passing e sincronização.
- **Broadcast:** eficiente para comunicação em grupo.
- MPI oferece muitas operações coletivas → *scatter, gather, reduce*.

- **Operações coletivas** → *scatter*, *gather*, *reduce*.
  Exemplo

```
if rank == 0:
    valores = [10 * i for i in range(size)]
else:
    valores = None

meu_valor = comm.scatter(valores, root=0)
resultado = meu_valor + rank # cada rank soma o valor
todos = comm.gather(resultado, root=0) # junta os resultados no
rank 0

if rank == 0:
    print("Valores originais:", valores, flush=True)
    print("Resultados por rank:", todos, flush=True)
```

- **Operações coletivas** → *scatter, gather, reduce*.
  Exemplo

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()

meu_numero = (rank + 1) * 7

soma = comm.reduce(meu_numero, op=MPI.SUM, root=0)
maximo = comm.reduce(meu_numero, op=MPI.MAX, root=0)

if rank == 0:
    print("Reduce SUM:", soma, flush=True)
    print("Reduce MAX:", maximo, flush=True)
```

MOM (Message-Oriented Middleware)

O que é MOM?

- Também chamado de **sistemas de filas de mensagens**.
- Oferecem suporte a **comunicação assíncrona persistente**.
- Diferem de sockets e MPI:
  - Não exigem que **emissor** e **receptor estejam ativos ao mesmo tempo**.
  - As mensagens podem demorar para serem entregues.

MOM – Modelo de Filas de Mensagens

- Aplicações se comunicam **inserindo mensagens em filas**.
- Mensagens são encaminhadas por **servidores de comunicação** até o destino.
- Se o destinatário estiver **offline**, a mensagem **permanece na fila** até que ele volte.
- Cada aplicação geralmente tem **sua própria fila privada**, mas é possível compartilhar filas entre múltiplas aplicações.

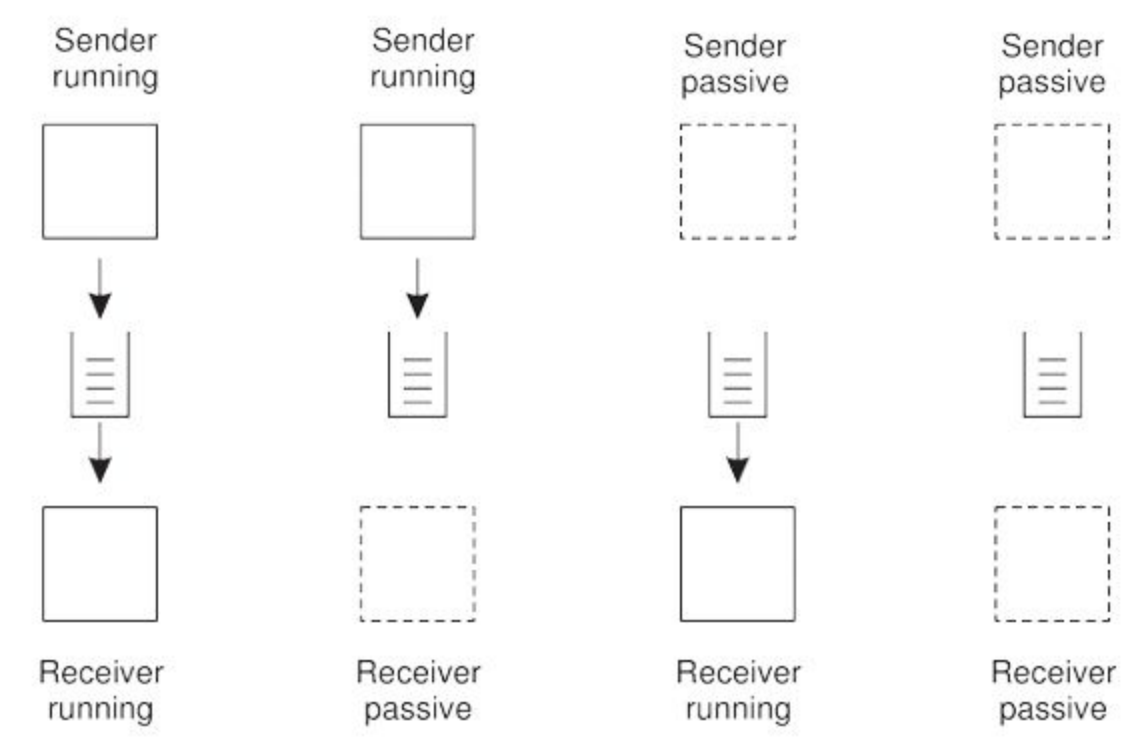
MOM - Modelo de Filas de Mensagens

- Não é necessário que **emissor e receptor** estejam ativos ao mesmo tempo.
- O envio e o recebimento podem acontecer em momentos distintos.
- Uma vez na fila, a mensagem **fica armazenada** até ser retirada.

MOM -Modos de Execução

- **Emissor ativo, receptor ativo** → mensagem enviada e lida imediatamente.
- **Emissor ativo, receptor inativo** → mensagem fica na fila até o receptor voltar.
- **Emissor inativo, receptor ativo** → receptor espera até que mensagens cheguem.
- **Emissor inativo, receptor inativo** → a fila guarda mensagens até ambos voltarem.

MOM -Modos de Execução



MOM -Modos de Execução

Exemplos de MOM modernos

- RabbitMQ
- Apache Kafka
- ActiveMQ
- Amazon SQS (AWS)

MOM - Operações básicas

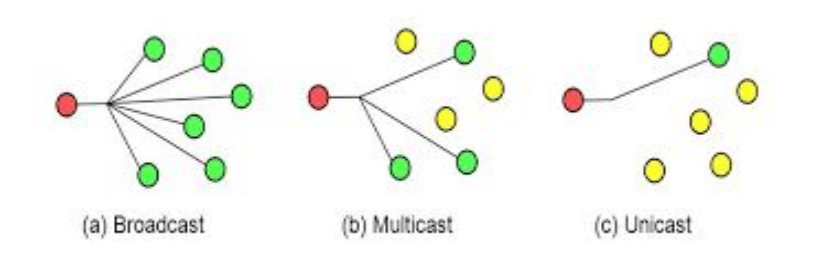
- PUT: Insere mensagem em uma fila.
- GET: Remove a primeira mensagem da fila.
  - Bloqueia se a fila estiver vazia
- POLL: Versão não bloqueante do GET.
  - Se não houver mensagem, retorna imediatamente.
- NOTIFY: Registra um callback (função tratadora).
  - Chamado automaticamente quando uma mensagem chega na fila.

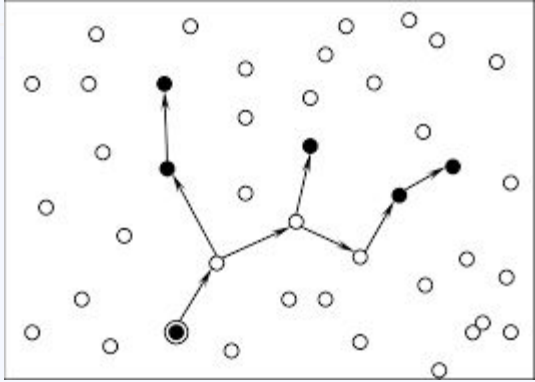
Multicast communication

Multicast: envio de dados de **um emissor para vários receptores**.

Diferente de:

- **Unicast** → um emissor → um receptor.
- **Broadcast** → um emissor → todos os nós da rede.





Tree-based Multicast

- Nós organizados em **árvore de sobreposição**.
- Cada par de nós tem um **caminho único**.

- Vantagens:
  - Estrutura simples.
  - Baixa redundância.

- Desvantagens:
  - **Fragilidade** → se um nó falha, parte da árvore pode ficar inacessível até reorganização.

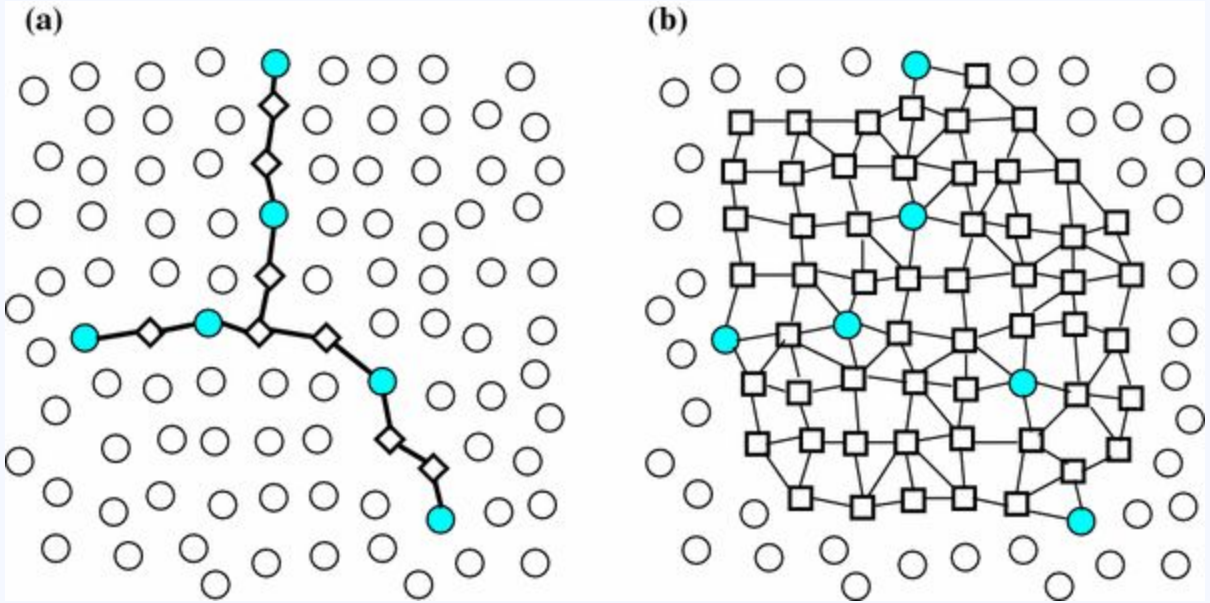
Mesh-based Multicast

- Nós formam uma **rede malha (mesh)**.
- Cada nó tem múltiplos vizinhos.
- Há **múltiplos caminhos** entre pares de nós.

- Vantagens:
  - **Alta robustez** (falha de um nó ≠ perda imediata de conectividade).
  - Tolerância a falhas.

- Desvantagens:
  - Mais **sobrecarga de mensagens** e consumo de largura de banda.

Multicast



Gossip-based Dissemination

- Alternativa simples ao multicast estruturado.
- Nós enviam mensagens para **um subconjunto aleatório de vizinhos**.
- Estes repassam para outros → efeito “boato” (*rumor spreading*).
- Vantagem: **robustez e simplicidade**.
- Desvantagem: **ineficiência** (muitas mensagens redundantes).

Flooding-based Multicasting

- **Multicast:** envio de mensagem a um **subconjunto** de nós (≠ broadcast para todos).

- **Flooding:** técnica ingênua em que **cada nó repassa a mensagem a todos os vizinhos**, exceto para o que enviou. Objetivo: garantir disseminação **sem conhecimento prévio** da topologia.

Problema do Flooding Puro

- Muitos nós encaminham mensagens para não destinatários.
- Gera altíssima redundância e tráfego excessivo.

Solução: **Probabilistic Flooding**

- Para reduzir o excesso: cada nó encaminha com probabilidade **pflood < 1**.
- Vantagens: Reduz dramaticamente o número de mensagens.
- Riscos: Possibilidade de **nós não receberem** a mensagem (se todos os vizinhos decidirem não encaminhar).